

Exclusão Mútua em Sistemas Distribuídos

Em sistemas distribuídos, **alguns recursos não podem ser utilizados simultaneamente** por vários processos — como, por exemplo, **arquivos, impressoras ou bancos de dados compartilhados**.

- Nesses casos, é necessário que o sistema assegure a **exclusividade de acesso**, conhecida como **exclusão mútua**.

Objetivo

Um **algoritmo de exclusão mútua** deve coordenar o acesso aos recursos compartilhados de modo que **apenas um processo por vez** possa utilizar, evitando conflitos e inconsistências.

A **exclusão mútua distribuída** é uma variação do problema clássico de exclusão mútua, adaptada para **sistemas distribuídos**.

A principal diferença em relação aos **sistemas centralizados** (como os sistemas operacionais tradicionais) é que, em ambientes distribuídos, **não existe memória compartilhada**.

Assim, **toda a sincronização e coordenação** entre processos deve ser realizada **por meio do envio e recebimento de mensagens**.

Requisitos de um algoritmo de exclusão mútua

1. **Exclusão mútua:**
Dado um recurso compartilhado que pode ser solicitado simultaneamente por vários processos, **somente um processo** deve ter permissão para acessá-lo em um determinado momento.
2. **Ausência de inanição (no-starvation):**
Todo processo que solicita acesso ao recurso **deve eventualmente obtê-lo**. Nenhum processo pode ficar indefinidamente esperando, ou seja, o algoritmo deve garantir **justiça no atendimento** das requisições.

Seção Crítica

- Chamamos de **seção crítica** a parte do código de um processo em que ocorre o **acesso a um recurso compartilhado**.
- Durante a execução da seção crítica, o processo precisa de **acesso exclusivo**, e o algoritmo de exclusão mútua é responsável por **assegurar que nenhum outro processo entre nessa seção ao mesmo tempo**.

Relação com o Problema de Acordo (*Agreement Problem*)

A exclusão mútua distribuída pode ser vista como um **caso particular do problema de acordo**, no qual **todos os processos precisam concordar** sobre **qual processo tem o direito de entrar na seção crítica** em determinado momento.

Em termos lógicos, os processos cooperam para atualizar uma variável booleana local, por exemplo:

```
posso_entrar SC = TRUE ou FALSE
```

Essa variável representa a **autorização** para entrar na seção crítica.

Definição do Problema

Considere N **processos** P_i , com $i=1, 2, \dots, N$, que se comunicam por troca de mensagens e precisam **sincronizar o acesso exclusivo** a um recurso compartilhado (ou a múltiplos recursos, de forma análoga).

O pseudocódigo típico de cada processo é:

```
enter();      // solicita entrada na seção crítica  
  
// acesso exclusivo ao recurso compartilhado  
exit();      // libera a seção crítica
```

O objetivo é garantir que **somente um processo** acesse a seção crítica simultaneamente.

Considerações sobre falhas

Nos algoritmos de exclusão mútua distribuída, **não é viável tolerar falhas de nós.**

Isso ocorre porque todos os processos precisam **chegar a um consenso** sobre **qual processo** entrará na seção crítica.

Se um dos participantes falhar, o acordo pode **não ser atingido**, comprometendo a consistência.

Premissas Comuns

Para o funcionamento correto dos algoritmos de exclusão mútua distribuída, geralmente assumem-se as seguintes condições:

- **Os processos não falham** (não há *crash* nem comportamento incorreto);
- **A comunicação é confiável** (todas as mensagens são entregues corretamente);
- **As mensagens não são duplicadas nem corrompidas;**
- **O sistema é assíncrono**, ou seja, **não há garantias sobre o tempo de transmissão** das mensagens nem sobre o tempo de execução dos processos;
- **Os processos não precisam saber** qual outro processo está, no momento, dentro da seção crítica.

Requisitos de um Algoritmo de Exclusão Mútua Distribuída

- Os algoritmos de **exclusão mútua distribuída** devem atender a certos **requisitos fundamentais**, divididos em **essenciais** e **opcionais**, para garantir o funcionamento correto e eficiente do sistema.

Requisitos Essenciais

1. **Segurança (Safety):** em qualquer instante, **no máximo um processo** pode estar executando sua **seção crítica**.
 - Isso assegura a **exclusividade de acesso** ao recurso compartilhado, evitando conflitos e inconsistências.
2. **Vivacidade (Liveness):** todo **pedido de entrada ou saída** da seção crítica deve ser **atendido em algum momento**.
 - Assim, evita-se tanto o **deadlock** (situação em que nenhum processo avança) quanto o **starvation** (quando um processo nunca é atendido).

Requisito Opcional

3. **Atendimento em Ordem:** as **solicitações de entrada** na seção crítica são tratadas **de acordo com a ordem causal** em que foram feitas.
 - Isso implica que, se um evento **A** ocorreu antes de outro **B**, o processo associado a **A** deve receber prioridade.
 - Esse comportamento garante **tratamento justo (fairness)**, permitindo que os processos acessem o recurso de forma ordenada e previsível.

Na implementação da **exclusão mútua em sistemas distribuídos**, existem **três principais abordagens** utilizadas para coordenar o acesso aos recursos compartilhados:

1. **Algoritmos baseados em *token***
2. **Algoritmos baseados em *timestamps***
3. **Algoritmos baseados em votação (*voting*)**



Algoritmos Baseados em *Token*

Nessa abordagem, uma **mensagem especial chamada token** é **circulada entre os processos** participantes do sistema.

- O **processo que possui o token** tem o **direito de entrar na seção crítica** e acessar o recurso compartilhado.
- Quando o processo **termina sua execução na seção crítica**, ele **repassa o token** para outro processo que esteja aguardando permissão para entrar.
- Dessa forma, o token atua como um **“passe exclusivo”** que concede temporariamente o direito de uso do recurso.

Tradução dos Requisitos Essenciais

- **Segurança (Safety):** deve **existir apenas um token** em todo o sistema. Isso garante que **somente um processo por vez** possa acessar a seção crítica.
- **Vivacidade (Liveness):** o token deve ser **capaz de alcançar qualquer processo** participante da sincronização.
 - Assim, todos os processos terão eventualmente a oportunidade de obter o token e acessar a seção crítica.

Estrutura Lógica de Comunicação

Para que o token seja transmitido de forma ordenada e eficiente, os algoritmos dessa categoria **definem uma estrutura lógica de comunicação** entre os processos.

As topologias mais comuns são:

- **Anel (ring):** os processos são organizados logicamente em um círculo, e o token é passado de um para outro na sequência.
- **Árvore (tree):** os processos são organizados em forma hierárquica, permitindo que o token seja encaminhado seguindo relações de pai e filho.

Essas estruturas facilitam o **controle e rastreamento do token**, reduzindo ambiguidades e garantindo a **coerência da exclusão mútua** no ambiente distribuído.

Algoritmo do Token Circulante

- O **algoritmo do token circulante** é uma das implementações clássicas da exclusão mútua distribuída **baseada em token**.
- Neste modelo, os processos são organizados em uma **estrutura lógica em anel**, por onde o token é **passado sequencialmente** entre todos os participantes da sincronização.

Modelo de Sistema (Premissas)

O funcionamento correto do algoritmo depende das seguintes condições:

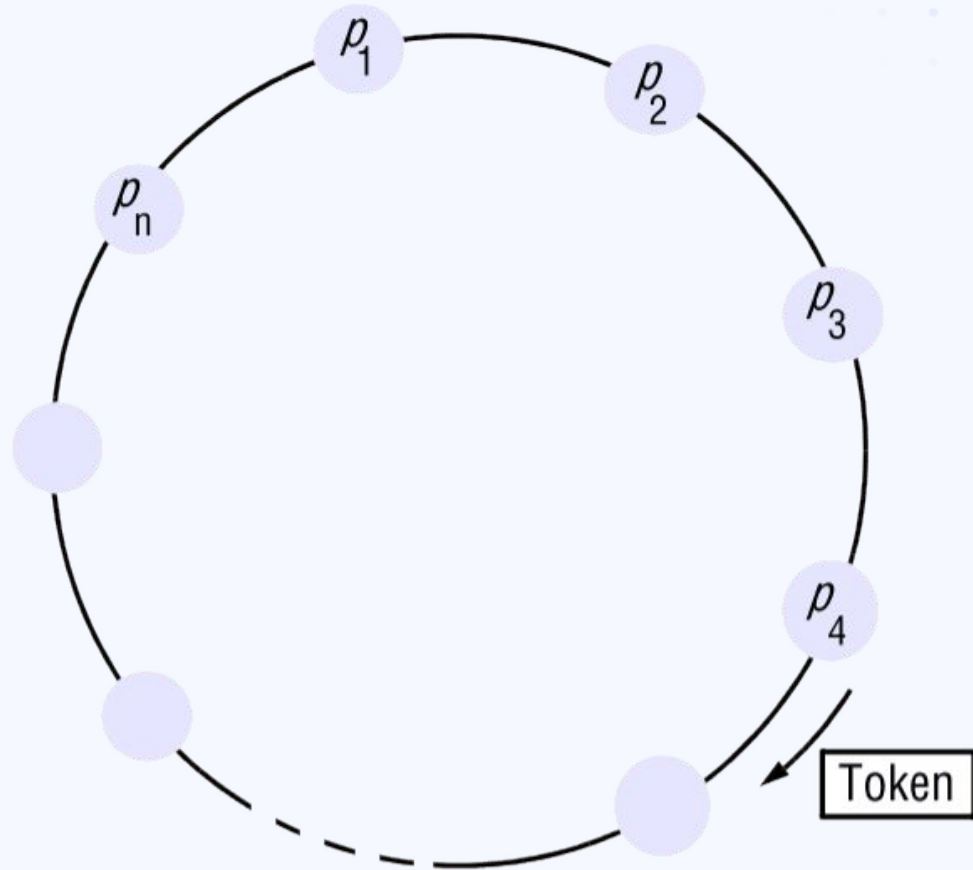
- A execução é **assíncrona** (não há relógio global nem tempos de espera fixos).
- A **topologia de interconexão é fixa**, organizada em **anel lógico**.
- O **número de processos é arbitrário**, mas **fixo** durante a execução.
- A **comunicação é confiável** — o token **nunca se perde**.
- A **comunicação é segura** — o token **não é duplicado nem alterado**.
- **Nenhum processo falha** durante a execução.

Funcionamento

Cada processo P_i do anel segue as seguintes regras:

1. Quando deseja entrar na **seção crítica**, o processo **aguarda a chegada do token** proveniente de seu **vizinho anterior** (à esquerda ou à direita, conforme a direção definida do anel).
2. Assim que recebe o token, o processo **entra na seção crítica** e **mantém o token consigo** enquanto executa suas operações exclusivas.
3. Ao **sair da seção crítica**, o processo **passa o token** ao **próximo processo** no anel (isto é, ao vizinho do qual **não** recebeu o token).
4. Se o processo **não precisa acessar** a seção crítica, ele **repassa o token imediatamente** ao próximo da sequência.

Algoritmo Token Ring



Propriedades

Atendidas as premissas acima, o algoritmo **garante**:

- **Segurança (Safety):** há **somente um token** no sistema, logo, apenas **um processo por vez** pode estar na seção crítica.
- **Vivacidade (Liveness):** o token **eventualmente alcança todos os processos**, assegurando que cada um possa executar sua seção crítica quando requisitado.

No entanto, o algoritmo **não garante atendimento em ordem causal**, pois o token circula independentemente da ordem das solicitações.

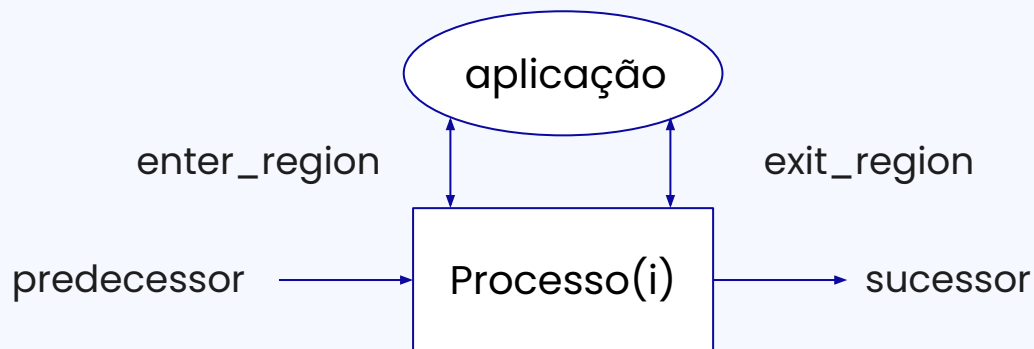
Observações

- O algoritmo possui **simetria textual**, ou seja, todos os processos executam **o mesmo código** e desempenham o **mesmo papel lógico**.
- Contudo, o **envio contínuo do token** gera um **overhead fixo** — o token circula **mesmo quando não há requisições** para entrar na seção crítica.
 - Isso significa que o custo de comunicação **não depende da frequência de pedidos**, o que pode tornar o método menos eficiente em períodos de baixa atividade.

Algoritmos Distribuídos – Algoritmo do Token Circulante

Cada processo **i** controla o **repasso do token** e atende a sua **própria aplicação local**:

- Recebe da aplicação os pedidos **enter_region** (entrar na SC) e **exit_region** (sair da SC).
- Recebe do **predecessor** no anel (**prev**) o **token circulante**.
- Mantém um estado booleano **waiting** indicando se há pedido pendente para entrar na seção crítica (SC).



Estados:
Waiting
~waiting

```
bool waiting = false

// Pedido local para entrar na SC
when recv(aplicacao, enter_region) and not waiting do
    waiting = true

// Chegada do token vindo do predecessor no anel
when recv(predecessor, token) do
    if waiting then
        // Concede entrada na SC à aplicação local
        reply(aplicacao, enter_region)      // sinaliza "pode entrar"
        recv(aplicacao, exit_region)        // aguarda sair da SC
        waiting = false
    end if

    // Repasse do token ao próximo no anel
    send(sucessor, token)
```


Problema

- Se o **token for perdido** (por exemplo, devido a erro de transmissão, ruído, ou falha de rede), o sistema **pode parar indefinidamente**, já que nenhum processo mais receberá o token — e, portanto, **ninguém poderá entrar na seção crítica**.
- O sistema precisa ser capaz de **detectar automaticamente** que o token se perdeu.

Algoritmo de Misra (1983)

A solução é **inserir um segundo token**, chamado **token complementar**, de modo que o sistema sempre mantenha **dois tokens circulando**:

- t_1 : token **principal**, que controla o **acesso à seção crítica**;
- t_2 : token **complementar**, que tem **valor oposto** a t_1 ($t_2 = -t_1$).

Regras do algoritmo

1. **Dois tokens circulam pelo anel no mesmo sentido.**
 - t_1 (principal) é usado normalmente para o controle da seção crítica.
 - t_2 (complementar) é apenas um **token de verificação**.
2. **Cada token carrega um valor numérico inteiro.**
 - No início, $t_1=0$ e $t_2=-t_1$.
 - Sempre que **os dois tokens se encontram** em um mesmo processo, eles são **atualizados**:
$$t_1 = t_1 + 1, \quad t_2 = t_2 - 1$$
 - Assim, eles ficam sempre **com valores opostos**: $t_2 = -t_1$

Detecção de perda:

- Se um processo ***P*** receber **qualquer um dos tokens (t_1 ou t_2)** com **o mesmo valor** que já havia recebido anteriormente, isso significa que **o outro token não está mais circulando**.
- Intuitivamente: se o valor não mudou, é porque o outro token **nunca encontrou este** para atualizar o par.

Recuperação (recriação do token perdido):

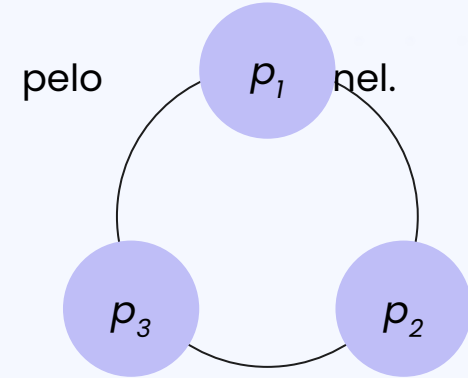
- O **primeiro processo** que detecta essa situação é, justamente, o processo que **deveria ter recebido o token perdido**.
- Esse processo, então, **recria o token faltante** (com o valor oposto) e o reinjeta na circulação.

$$t_1 = 0$$

$$t_2 = 0$$

Exemplo: imagine um anel com três processos P_1, P_2, P_3 .

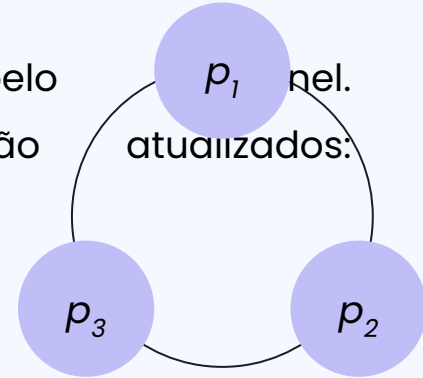
1. t_1 e t_2 circulam



Algoritmos Distribuídos – Algoritmo do Token Circulante

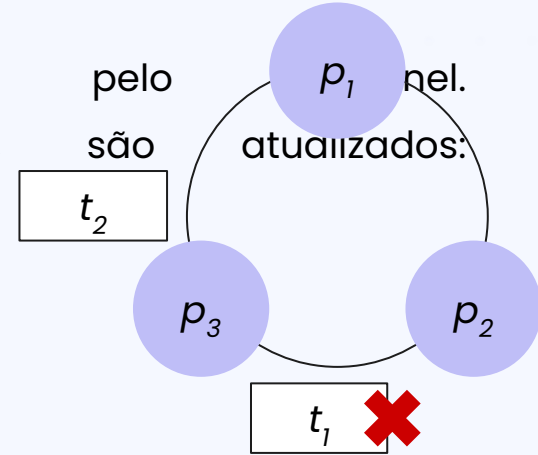
Exemplo: imagine um anel com três processos P_1, P_2, P_3 .

1. t_1 e t_2 circulam pelo anel.
Quando ambos passam por P_1 , são atualizados:
 $t_1 = 1, t_2 = -1$



Exemplo: imagine um anel com três processos P_1, P_2, P_3 .

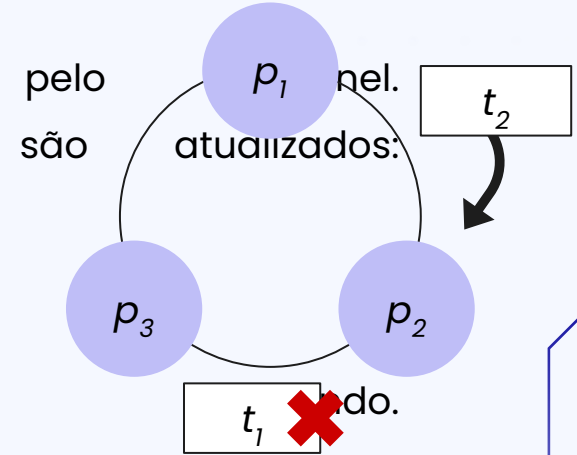
- t_1 e t_2 circulam
Quando ambos passam por P_1 ,
 $t_1 = 1, t_2 = -1$
- Por falha de transmissão, t_1 se perde entre P_2 e P_3 .



Algoritmos Distribuídos – Algoritmo do Token Circulante

Exemplo: imagine um anel com três processos P_1, P_2, P_3 .

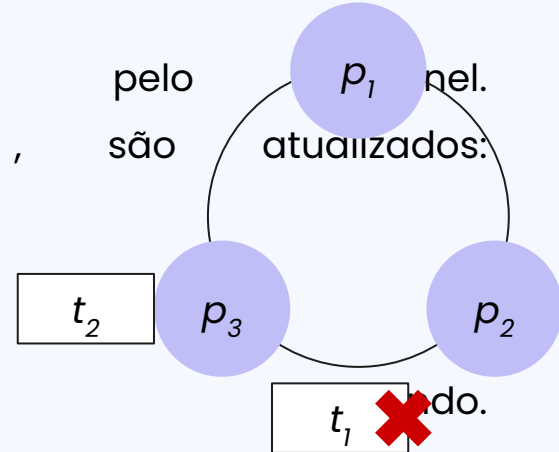
1. t_1 e t_2 circulam
Quando ambos passam por P_1 ,
 $t_1 = 1, t_2 = -1$
2. Por falha de transmissão, t_1 se perde entre P_2 e P_3 .
3. t_2 continua



Algoritmos Distribuídos – Algoritmo do Token Circulante

Exemplo: imagine um anel com três processos P_1, P_2, P_3 .

- t_1 e t_2 circulam pelo anel.
Quando ambos passam por P_1 , são atualizados:
 $t_1 = 1, t_2 = -1$
- Por falha de transmissão, t_1 se perde entre P_2 e P_3 .
- t_2 continua
Quando P_3 recebe t_2 com o mesmo valor anterior, ele percebe:
"O outro token (t_1) não foi atualizado, logo se perdeu."



Algoritmos Distribuídos – Algoritmo do Token Circulante

Exemplo: imagine um anel com três processos P_1, P_2, P_3 .

1. t_1 e t_2 circulam pelo anel.
Quando ambos passam por P_1 , são atualizados:
 $t_1 = 1, t_2 = -1$

2. Por falha de transmissão, t_1 se perde entre P_2 e P_3 .

3. t_2 continua circulando.
Quando P_3 recebe t_2 com o mesmo valor anterior, ele percebe:
"O outro token (t_1) não foi atualizado, logo se perdeu."

4. P_3 então **recria o token perdido:**
 $t_1 = -t_2$

e o sistema **volta a operar normalmente.**

Algoritmos Distribuídos – Algoritmo do Token Circulante

```
when recv(i-1, t1) {
    if (mi == t1.val) {          // t2 provavelmente foi perdido
        t1.val = t1.val + 1;
        mi = t1.val;

        new(t2);
        t2.val = -t1.val;
        send(i+1, t2);
    }
    else {
        mi = t1.val;
    }

    if (waiting) {
        entra_na_secao_critica();
        ao_sair_send(i+1, t1);
    }
    else {
        send(i+1, t1);
    }
}
```

Seja mi uma variável em P_i que guarda o último valor de um token ($t1$ ou $t2$) visto.

O atributo $t.val$ armazena o contador associado ao token.

Algoritmos Distribuídos – Algoritmo do Token Circulante

```
when recv(i-1, t2) {
    if (mi == t2.val) {          // t1 provavelmente foi perdido
        t2.val = t2.val - 1;
        mi = t2.val;

        new(t1);
        t1.val = -t2.val;
        send(i+1, t1);
    }
    else {
        mi = t2.val;
    }

    send(i+1, t2);              // t2 não entra na seção crítica
}

when encontro(t1, t2) {
    t1.val = t1.val + 1;
    t2.val = -t1.val;
}
```

Seja mi uma variável em P_i que guarda o último valor de um token ($t1$ ou $t2$) visto.

O atributo $t.val$ armazena o contador associado ao token.

Algoritmos Distribuídos – Algoritmo do Token Circulante

Para evitar falsa detecção de perda de token, os valores absolutos assumidos por t_1 e t_2 precisam ser diferentes de todos os valores m_i , e seus incrementos precisam ser no mínimo mod $N+1$ (pois tokens podem se encontrar até N vezes).

Algoritmo do Token em Árvore

O objetivo principal do algoritmo do Token em Árvore é **evitar o envio desnecessário do token** (como ocorre na topologia em **anel**, onde ele circula continuamente, mesmo sem haver pedidos para a seção crítica).

- a **raiz da árvore** é sempre o **processo que possui o token**.

Funcionamento:

1. O **processo que possui o token** é considerado **a raiz** da árvore lógica.
2. Cada processo mantém uma **referência curr_dir** (current direction) — ou seja, o **vizinho mais próximo do nó raiz** (o caminho “na direção” do token).
3. Quando o **token é transferido**, a **estrutura da árvore é atualizada dinamicamente** — o novo detentor se torna a nova raiz.

Etapas

1. Pedido para entrar na seção crítica

Quando um processo P_i quer entrar na **seção crítica (SC)**, ele:

- Envia uma mensagem **REQ (request)** ao vizinho indicado em sua variável **curr_dir** — ou seja, o processo que ele acredita estar **mais próximo do token**.

Etapas

2. Propagação da requisição

Ao receber uma mensagem **REQ**, o processo vizinho verifica:

- Se **não tem o token** e **não enviou ainda um pedido anterior**, ele:
 - **Encaminha um novo REQ** para o seu próprio **curr_dir** (ou seja, sobe um nível em direção ao token);
 - Marca que **já tem um pedido pendente** (para não reenviar mais requisições duplicadas).

Assim, o pedido **propaga-se pela árvore** até chegar ao **processo que possui o token (a raiz atual)**.

Etapas

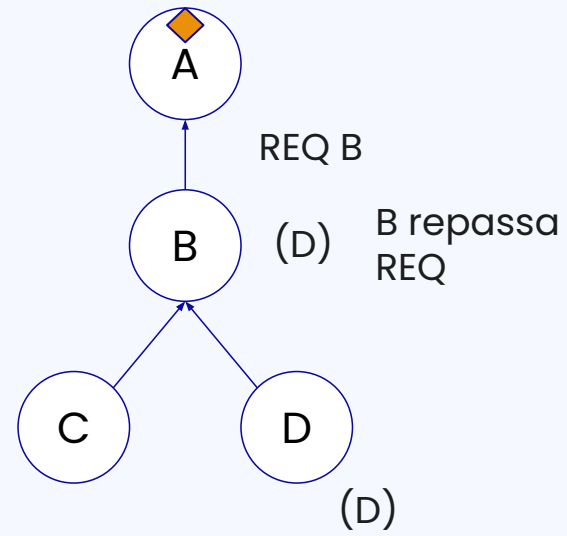
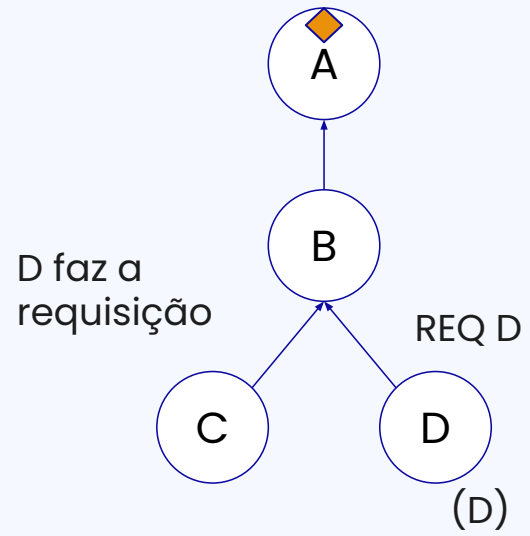
3. Armazenamento de requisições pendentes

Cada processo intermediário **mantém uma fila de requisições pendentes**, onde **a cabeça da fila** indica **o caminho pelo qual o token deve seguir** quando for transferido.

- Quando o processo recebe o token e verifica pedidos pendentes, ele o **envia para o primeiro processo da fila** (aquele mais próximo da requisição original).
- Com isso, o **token se move ao longo da árvore**, criando um novo caminho e atualizando as referências `curr_dir` nos nós intermediários.

Exemplo

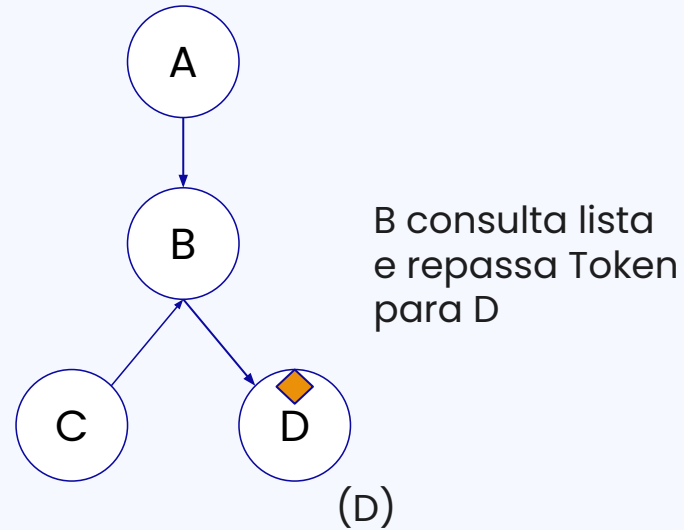
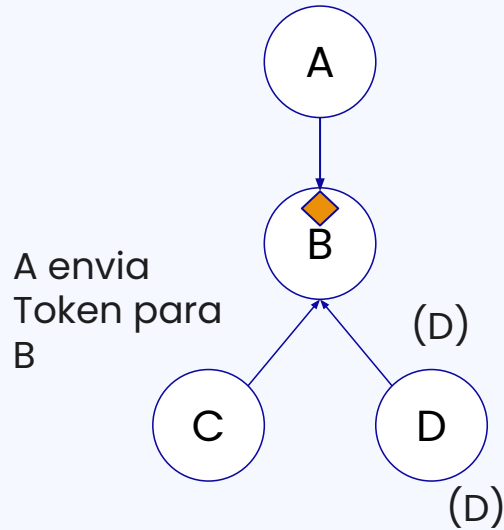
→ representa curr_dir,
(X) requisição pendente do processo X



Algoritmos Distribuídos – Algoritmo do Token em Árvores

Exemplo

→ representa curr_dir,
(X) requisição pendente do processo X



Algoritmos Distribuídos – Algoritmo do Token em Árvores

Algoritmo de Raymond

```
when EnterCS {
    // primeiro: registra o pedido local
    if (me not in reqQ) {
        reqQ.add(me);
    }

    // se tenho o token e sou o primeiro da
    // fila,
    if (token == true && reqQ.first() == me) {
        reqQ.rem();
        InCS = true;
        return;
    }

    // se tenho o token, mas tem outro pedido
    // antes do meu
    if (token == true && reqQ.first() != me) {
        next = reqQ.rem();

        curr_dir = next;
        send(next, TOKEN);
        token = false;
        asked = false;
```

```
    // como meu pedido ainda está na fila,
    // peço o token novamente
    if (not empty(reqQ)) {
        send(curr_dir, REQ);
        asked = true;
    }
}

// se eu não tenho o token e ainda não pedi
if (token == false && asked == false) {
    send(curr_dir, REQ);
    asked = true;
}

// espera o token chegar e meu pedido ser o
// primeiro
wait until token == true && not empty(reqQ)
&& reqQ.first() == me;

reqQ.rem();
InCS = true;
return;
}
```

Variáveis em Pi:

```
bool token // true se Pi possui o token
bool InCS // true se Pi está na seção crítica
bool asked // true se Pi já enviou REQ para curr_dir
Addr curr_dir // vizinho na direção do detentor do
token
AddrQueue reqQ // fila de pedidos pendentes
```

Algoritmo de Raymond

```
when ExitCS {  
    InCS = false;  
  
    if (empty(reqQ)) {  
        return;  
    }  
  
    next = reqQ.rem();  
  
    curr_dir = next;  
    send(next, TOKEN);  
    token = false;  
    asked = false;  
  
    if (not empty(reqQ)) {  
        send(curr_dir, REQ);  
        asked = true;  
    }  
}
```

Considere um sistema distribuído com **N processos**, interligados em uma **topologia de grafo completo** (ou seja, todos os processos podem se comunicar diretamente entre si).

Funcionamento:

1. Um processo **só pode entrar na seção crítica (SC)** se **receber o consentimento de todos os outros processos (consenso)**.
2. Para isso, cada processo **difunde (broadcast)** sua requisição de entrada na SC a todos os demais processos.
3. Cada requisição é **marcada com um timestamp lógico**, calculado segundo o **relógio de Lamport**, permitindo que todos os processos **ordenem globalmente os pedidos**.
4. Assim, cada processo sabe **quem requisitou primeiro**, respeitando a **ordem causal total** das requisições.

Regras de decisão

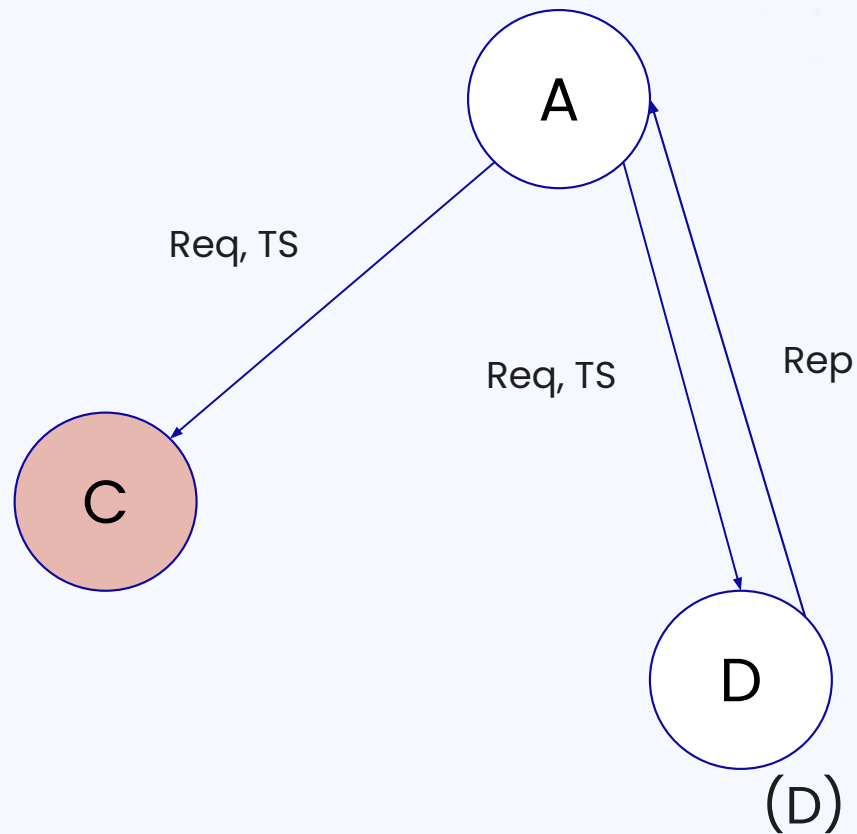
Quando um processo P_i recebe um pedido de outro processo P_r contendo uma marca de tempo t_s , ele decide se responde **imediatamente** ou **posterga a resposta**, conforme:

- Se P_i **não está na seção crítica e não tem nenhum pedido próprio pendente anterior** (com timestamp menor), então ele **envia imediatamente um REPLY (OK)** para P_r .
- Caso contrário, se:
 - já está executando sua seção crítica, ou
 - possui um pedido próprio com **timestamp menor** (isto é, anterior na ordem lógica), então ele **adia o envio do REPLY** até que sua vez tenha terminado.

Cada requisição é ordenada segundo o **timestamp lógico de Lamport**, que garante uma **ordem total coerente** entre todos os processos.

- Assim, se **dois processos requisitarem ao mesmo tempo**, apenas **um deles** receberá os **N-1 REPLYs** necessários para entrar na SC primeiro.
- O outro processo aguardará até que o primeiro saia da SC e **libere suas respostas pendentes**.

Algoritmos Distribuídos – Algoritmos Baseados em Timestamps



C está na SC e
adia a resposta

Algoritmos Baseados em Votação (Quorum)

Os **algoritmos de exclusão mútua baseados em votação**, também conhecidos como **algoritmos de *quorum***, seguem a ideia de que **nem todos os processos precisam ser consultados** para autorizar a entrada de um processo na seção crítica (SC).

Em vez disso, apenas **um subconjunto representativo** é suficiente — semelhante a uma **eleição política**, em que **um pequeno grupo de eleitores “decide” o resultado final**.

Funcionamento

1. Quando um processo deseja entrar na seção crítica, ele **não precisa solicitar permissão de todos os outros N processos**.
2. Em vez disso, ele **consulta apenas um subconjunto** de processos (**distrito**).
3. Se o processo **receber confirmação (OK)** de **todos os membros do distrito**, ele **pode entrar na SC**.
4. Caso contrário, **aguarda** até receber todas as respostas necessárias.

Vantagem principal: reduz significativamente o **número total de mensagens trocadas**.

Algoritmos Distribuídos – Algoritmos Baseados em Votação (Quorum)

O algoritmo proposto por **Maekawa (1985)** é o exemplo clássico dessa abordagem.

Seja $\{P_1, P_2, \dots, P_M\}$ o conjunto de processos

- A cada processo P_i é associado um **distrito S_i** , que é um **subconjunto de processos** do sistema.

Esses distritos precisam satisfazer as seguintes propriedades:

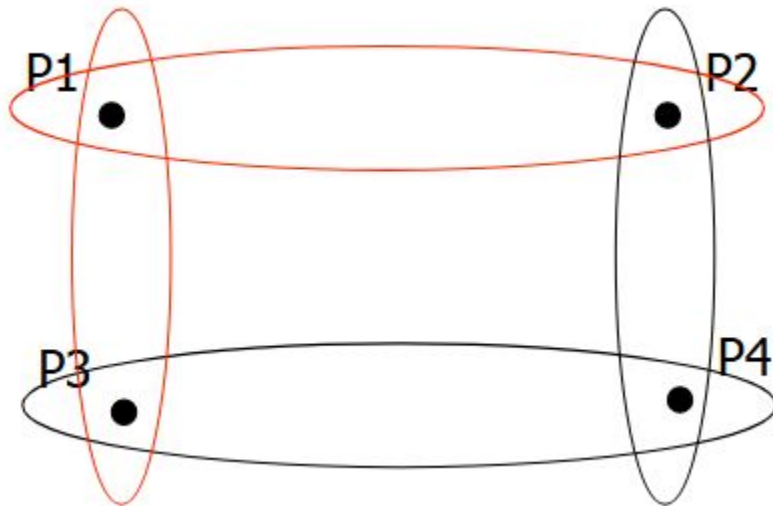
1. **Inclusão própria:** cada processo pertence ao seu próprio distrito.
2. **Interseção obrigatória:** qualquer par de distritos possui pelo menos um processo em comum.
 - a. **A fim de que dois processos diferentes nunca possam entrar na SC simultaneamente,** pois ambos teriam de obter o voto do mesmo processo.
3. **Tamanho uniforme (quando possível):** todos os distritos devem ter aproximadamente o mesmo número de elementos.
4. **Distribuição equilibrada:** cada processo P_j **pertence a D distritos** (isto é, participa como eleitor em D subconjuntos distintos).
5. **Não inclusão total:** nenhum distrito S_i deve ser **subconjunto completo** de outro.

Algoritmos Distribuídos – Algoritmos Baseados em Votação (Quorum)

Passos:

1. Quando P_i quer entrar na SC:
 - Envia mensagens **REQUEST** a todos os processos de seu distrito S_i .
2. Cada processo em S_i :
 - Responde **OK** se não tiver concedido voto a outro processo no momento;
 - Caso contrário, **adiará** a resposta até liberar o voto.
3. Quando P_i **recebe todos os OKs** do seu distrito S_i :
 - Entra na seção crítica.
4. Ao sair da SC:
 - Envia mensagens **RELEASE** para liberar os votos obtidos.

Algoritmos Distribuídos – Algoritmos Baseados em Votação (Quorum)



$S1 = \{p1, p2, p3\}$

$S2 = \{p1, p2, p4\}$

$S3 = \{p1, p3, p4\}$

$S4 = \{p2, p3, p4\}$